

Hafta 9

Çizge Algoritmaları

Sıralama · Ağırlıklar · Döngüler · Geri İzleme

Bugünün haritası

- Sıralama: topolojik sıralama (2 yöntem), döngü tespiti
- Grupları takip et: union-find (ayrık küme)
- En ucuz bağlantılar: Kruskal, Prim (MST)
- En ucuz yollar: Dijkstra, Bellman-Ford, Floyd-Warshall
- Erişilebilirlik: güçlü bağlı bileşenler, iki parçalılık
- Yolların ötesi: en büyük akış, geri izleme

Başlamadan önce: bildikleriniz

- Hafta 5: komşuluk listesi, alfabetik komşular
- BFS: kuyruk, en az kenar sayısı
- DFS: özyineleme veya açık yığın
- Bağlı bileşenler: her ziyaret edilmemiş köşeden BFS/DFS

Tek yeni bileşen: ağırlıklar

- Bugünün algoritmalarından 6'sı her kenara bir **sayı** iliştirir
- Bir mesafe, bir maliyet, bir kapasite
- Soru "ulaşılabilir mi?"den ******"en ucuzu hangisi?"**ye değişir

Bu hafta Hafta 5'ten ne kullanıyor

Hafta 5 aracı	Kullanan
Komşuluk listesi, alfabetik sıra	Bu haftaki her algoritma
BFS kuyruğu	İki parçalılık kontrolü, en büyük akışın genişletme yolu
DFS özyinelemesi	Topolojik sıralama, döngü tespiti, GBB, geri izleme
Çember yerleşimi	Her animasyonun çizge çizimi

1. Topolojik Sıralama

Başlangıç sorusu

Çoraplar ayakkabılardan önce. Gömlek ceketten önce.
Çorap ile gömlek arasında bir sıra yok.

Her kuralı gözeten tek bir sıra her zaman var mıdır?

Kısa tarihçe

- **A. B. Kahn**, 1962 — içe-derece (in-degree) + kuyruk
- Build sistemleri, paket kurulumcuları bunu her gün kullanır
- DFS tabanlı alternatif, **Tarjan**'ın 1970'lerdeki DFS çerçevesinden çıkar

Kahn'ın fikri

- **İçe-derecesi 0** olan bir köşenin karşılanmamış ön koşulu yoktur
- Onu şimdi yerleştir; bu, giden kenarlarını kaldırır
- Her komşunun içe-derecesi bir azalır
- Yeni sıfırlanan komşular uygun hale gelir — onları kuyruğa al

Kahn'ın kodu (C) — bölüm 1/2

```
int topo_sort_kahn(Graph *g) {
    for (int i=0; i<g->vertex_count; i++) indeg[i]=0;
    for (int u=0; u<g->vertex_count; u++)
        for (AdjNode *n=g->adj[u]; n; n=n->next)
            indeg[n->to]++;
    front=rear=0; order_len=0;
    for (int v=0; v<g->vertex_count; v++)
        if (indeg[v]==0) enqueue(v);
}
```

Kahn'ın kodu (C) — bölüm 2/2

```
while (front<rear) {
    int u=dequeue();
    order[order_len++]=u;
    for (AdjNode *n=g->adj[u]; n; n=n->next) {
        indeg[n->to]--;
        if (indeg[n->to]==0) enqueue(n->to);
    }
}
return order_len==g->vertex_count;
}
```

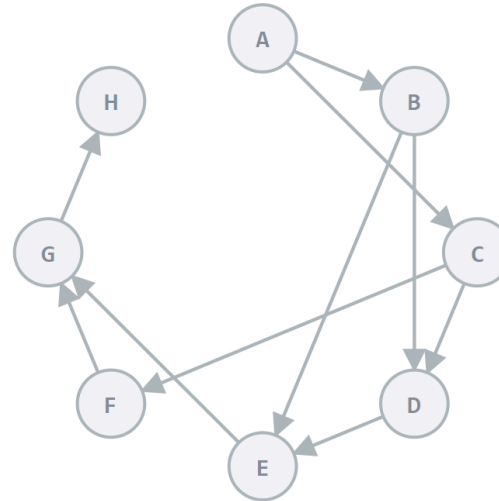
Kahn'ın kodu (Java) — bölüm 1/2

```
static boolean topoSortKahn(Graph g) {  
    for (int i=0; i<g.vertexCount; i++) indeg[i]=0;  
    for (int u=0; u<g.vertexCount; u++)  
        for (AdjNode n=g.adj[u]; n!=null; n=n.next)  
            indeg[n.to]++;  
    front=rear=0; orderLen=0;  
    for (int v=0; v<g.vertexCount; v++)  
        if (indeg[v]==0) enqueue(v);  
}
```

Kahn'ın kodu (Java) — bölüm 2/2

```
while (front < rear) {  
    int u = dequeue();  
    order[orderLen] = u; orderLen++;  
    for (AdjNode n = g.adj[u]; n != null; n = n.next) {  
        indeg[n.to]--;  
        if (indeg[n.to] == 0) enqueue(n.to);  
    }  
}  
return orderLen == g.vertexCount;  
}
```

Kahn'ın algoritması



ıçderece[] =

A	B	C	D	E	F	G	H
0	0	0	0	0	0	0	0

kuyruk =

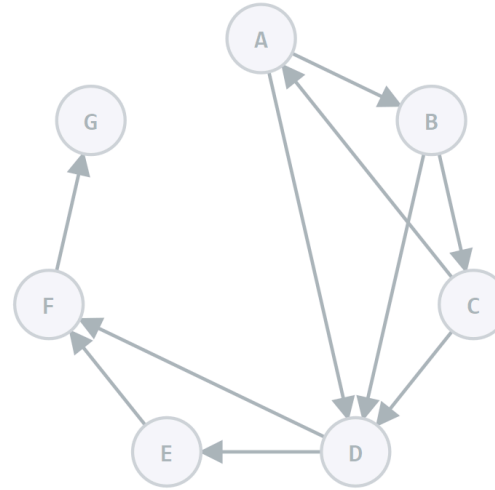
--	--	--	--	--	--	--	--

sıra =

A	B	C	D	F	E	G	H
---	---	---	---	---	---	---	---

Kuyruk boşaldı, 8 düğümün hepsi sıraya girdi: A, B, C, D, F, E, G, H. Her düğüm bütün önkoşullarından sonra gelir.

Uç durum: bir döngü



ıçderece[] =

A	B	C	D	E	F	G
1	1	1	3	1	2	1

kuyruk =

--	--	--	--	--	--	--

sıra =

Kuyruk boşaldı ama A, B, C, D, E, F, G hâlâ sıraya girmedi -- bunların hepsi içderecesi hiç 0 olmayan bir DÖNGÜNÜN parçası. Topolojik sıralama YOK.

Gerçek çıktı: Kahn'ın algoritması

```
-- normal: 8 vertices, 10 edges, a valid DAG --  
order: A B C D F E G H  
all 8 vertices placed: a valid topological order  
  
-- edge: 10 edges but a cycle exists, no full order --  
order:  
only 0 of 7 vertices placed -- a cycle exists
```

DFS fikri

- Özyinelemeli DFS çalıştır (Hafta 5'in DFS'i, bir dizi eklenmiş)
- Her köşenin **bitiş zamanını** kaydet
- Bitiş zamanlarını **büyükten küçüğe** oku
- Bir köşe, işaret ettiği her şeyden sonra biter

DFS topolojik sıralama kodu (C) — dfs_visit

```
void dfs_visit(int u) {  
    color_of[u]=1;  
    for (AdjNode *n=cur_g->adj[u]; n; n=n->next) {  
        if (color_of[n->to]==0) dfs_visit(n->to);  
        else if (color_of[n->to]==1) has_cycle=1;  
    }  
    color_of[u]=2;  
    finish[finish_len++]=u;  
}
```

DFS topolojik sıralama kodu (C) — sürücü

```
void topo_sort_dfs(Graph *g) {  
    cur_g=g;  
    for (int i=0;i<g->vertex_count;i++) color_of[i]=0;  
    finish_len=0; has_cycle=0;  
    for (int v=0;v<g->vertex_count;v++)  
        if (color_of[v]==0) dfs_visit(v);  
}
```

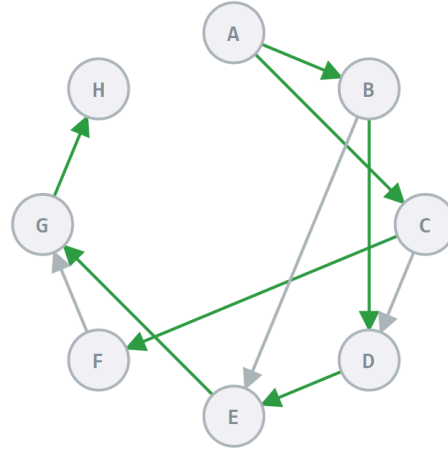
DFS topolojik sıralama kodu (Java) — dfsVisit

```
static void dfsVisit(int u) {  
    colorOf[u]=1;  
    for (AdjNode n=curG.adj[u]; n!=null; n=n.next) {  
        if (colorOf[n.to]==0) dfsVisit(n.to);  
        else if (colorOf[n.to]==1) hasCycle=true;  
    }  
    colorOf[u]=2;  
    finish[finishLen]=u; finishLen++;  
}
```

DFS topolojik sıralama kodu (Java) — sürücü

```
static void topoSortDfs(Graph g) {  
    curG=g;  
    for (int i=0;i<g.vertexCount;i++) colorOf[i]=0;  
    finishLen=0; hasCycle=false;  
    for (int v=0;v<g.vertexCount;v++)  
        if (colorOf[v]==0) dfsVisit(v);  
}
```

DFS topolojik sıralama



renk[] =

A	B	C	D	E	F	G	H
2	2	2	2	2	2	2	2

çağrı yığını =

--	--	--	--	--	--	--	--

bitiş sırası =

H	G	E	D	B	F	C	A
---	---	---	---	---	---	---	---

topo sıra =

A	C	F	B	D	E	G	H
---	---	---	---	---	---	---	---

bitiş sırası ters çevrilir: A, C, F, B, D, E, G, H. Her düğüm, kendisine gelen bütün kenarlardan sonra gelir.

Karmaşıklık ve tuzak

- $O(V + E)$ — her iki algoritma da, tek geçiş
- Tuzak: bir topolojik sıra **tek değildir**
- **Konumları** karşılaştırın, kesin diziyi asla sabit kodlamayın

Kısa soru

Kahn'ın algoritması neden tek bir özyinelemeli çağrı değil de bir **kuyruk** kullanır?

(cevap bir sonraki slaytta)

Cevap

Bir DAG'ın aynı anda **birden fazla bağımsız kaynağı** olabilir.

Kuyruk, algoritmanın bir dalı bitirmeden diğerine başlamak yerine, tek geçişte onları iç içe geçirmesini sağlar.

2. Döngü Tespiti (Yönlü)

Başlangıç sorusu

Bölüm 1'in DFS'i bir döngünün var **olduğunu** tespit eder (bir geri kenar).

Hangi köşeler, hangi sırayla onu oluşturuyor?

Yollu 3 renkli DFS

- Beyaz / **gri** (yol üzerinde) / siyah (bitmiş)
- Güncel yolu `on_path[]` içinde tut
- **Gri** bir köşeye giden bir geri kenar: o köşe açık bir atadır
- Oradan buraya kadarki yol dilimi döngünün **kendisidir**

Döngü tespiti kodu (C) — bölüm 1/2

```
int dfs_cycle(int u) {  
    color_of[u]=1;  
    on_path[path_top++]=u;  
    for (AdjNode *n=cur_g->adj[u]; n; n=n->next) {  
        if (color_of[n->to]==0) {  
            if (dfs_cycle(n->to)) return 1;  
        } else if (color_of[n->to]==1) {
```

Döngü tespiti kodu (C) — bölüm 2/2

```
        int i=path_top-1;
        while (on_path[i]!=n->to) i--;
        cycle_len=0;
        for (; i<path_top; i++)
            cycle[cycle_len++]=on_path[i];
        return 1;
    }
}
path_top--; color_of[u]=2;
return 0;
}
```

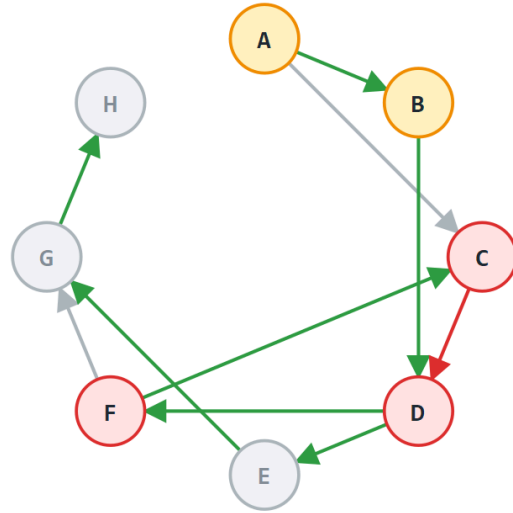
Döngü tespiti kodu (Java) — bölüm 1/2

```
static boolean dfsCycle(int u) {  
    colorOf[u]=1;  
    onPath[pathTop]=u; pathTop++;  
    for (AdjNode n=curG.adj[u]; n!=null; n=n.next) {  
        if (colorOf[n.to]==0) {  
            if (dfsCycle(n.to)) return true;  
        } else if (colorOf[n.to]==1) {
```

Döngü tespiti kodu (Java) — bölüm 2/2

```
        int i=pathTop-1;
        while (onPath[i]!=n.to) i--;
        cycleLen=0;
        for (; i<pathTop; i++) {
            cycle[cycleLen]=onPath[i]; cycleLen++;
        }
        return true;
    }
}
pathTop--; colorOf[u]=2;
return false;
}
```

Yönlü çizgede döngü tespiti



renk[] =

A	B	C	D	E	F	G	H
1	1	1	1	2	1	2	2

yol (on_path) =

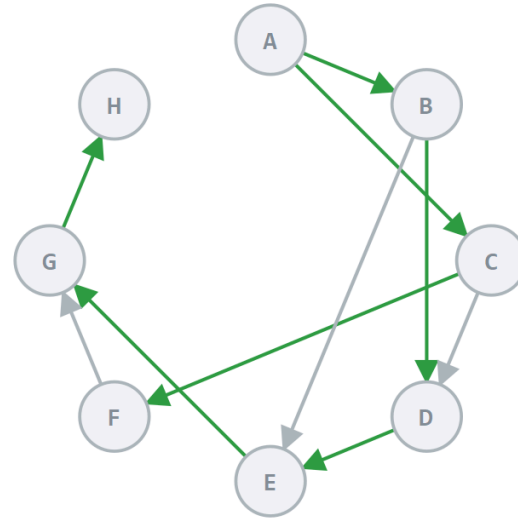
A	B	D	F	C			
---	---	---	---	---	--	--	--

döngü =

D	F	C
---	---	---

Komşu D gri: D hâlâ yolda, bir ATA. Yoldaki 2. konumdan buraya kadarki dilim BİR DÖNGÜ: D -> F -> C -> D.

Uç durum: hiç döngü yok



renk[] =

A	B	C	D	E	F	G	H
2	2	2	2	2	2	2	2

yol (on_path) =

--	--	--	--	--	--	--	--

döngü =

Bütün düğümler siyah oldu, hiç geri kenar bulunmadı: bu çizgede DÖNGÜ YOK.

Gerçek çıktı: döngü tespiti

```
-- normal: 8 vertices, 10 edges, one cycle: C-D-F-C --  
cycle found: D F C -> D  
  
-- edge: 10 edges, entirely cycle-free (a DAG) --  
no cycle found  
  
-- edge: the smallest cycle, A-B-A (2 edges) --  
cycle found: A B -> A
```

Karmaşıklık ve tuzak

- $O(V + E)$ — bir DFS, artı bulunan döngüyü çıkarmak için $O(V)$
- Tuzak: "**gri**" yerine "beyaz değil" kontrolü yapmak
- Siyah bir komşu bitmiştir ve güvenlidir — döngü değildir

Kısa soru

Bölüm 1'in DFS topolojik sıralaması sadece `has_cycle = 1` yapıyor. Bu algoritma neden ekstra bir `on_path[]` yığınınına ihtiyaç duyuyor?

(cevap bir sonraki slaytta)

Cevap

Bir döngünün var olduğunu bilmek, onu **hangi köşelerin** oluşturduğunu bilmekle aynı değildir.

`on_path[ ]`, güncel kök-buraya zincirini erişilebilir tutar; böylece bir geri kenar bulunduğu anda döngü doğrudan ondan okunabilir.

3. Union-Find (Ayrık Küme)

Başlangıç sorusu

Kruskal'ın algoritması (birazdan geliyor) tekrar tekrar şunu sormalı:

"Bu iki köşe, seçtiğim kenarlarla zaten bağlı mı?"

Her seferinde taze bir BFS doğru olur — ama yavaş olur.

Ebeveyn işaretçilerinden bir orman

- `find(v)`, ebeveyn işaretçilerinde **köke** kadar yürür
- Aynı kök = aynı grup
- `union(a, b)`, iki grubu birleştirir: kök köke işaret eder

İki hile

- **Ranka göre birleştirme:** kısa ağaç, uzun olanın altına asılır
- **Yol sıkıştırma:** `find`, ziyaret ettiği her köşeyi doğrudan köke yeniden bağlar
- Birlikte: işlem başına **$O(\alpha(n))$** — pratikteki her `n` için bu $O(1)$ 'dir

Union-find kodu (C) — find

```
int find(int v) {  
    int root=v;  
    while (parent_of[root]!=root) root=parent_of[root];  
    while (parent_of[v]!=root) {  
        int next=parent_of[v];  
        parent_of[v]=root;  
        v=next;  
    }  
    return root;  
}
```

Union-find kodu (C) — union_sets

```
void union_sets(int a, int b) {  
    int ra=find(a), rb=find(b);  
    if (ra==rb) return;  
    if (rank_of[ra]<rank_of[rb]) parent_of[ra]=rb;  
    else if (rank_of[ra]>rank_of[rb]) parent_of[rb]=ra;  
    else { parent_of[rb]=ra; rank_of[ra]++; }  
}
```

Union-find kodu (Java) — find

```
static int find(int v) {  
    int root=v;  
    while (parentOf[root]!=root) root=parentOf[root];  
    while (parentOf[v]!=root) {  
        int next=parentOf[v];  
        parentOf[v]=root;  
        v=next;  
    }  
    return root;  
}
```

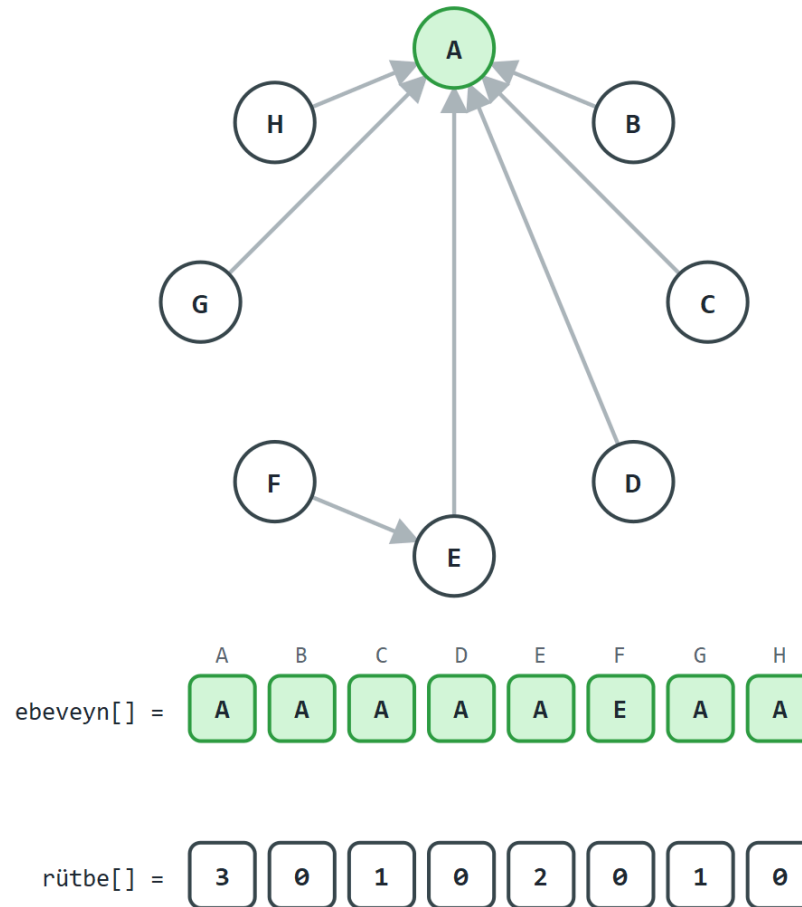
Union-find kodu (Java) — unionSets

```
static void unionSets(int a, int b) {  
    int ra=find(a), rb=find(b);  
    if (ra==rb) return;  
    if (rankOf[ra]<rankOf[rb]) parentOf[ra]=rb;  
    else if (rankOf[ra]>rankOf[rb]) parentOf[rb]=ra;  
    else { parentOf[rb]=ra; rankOf[ra]++; }  
}
```

Gerçek çıktı: union-find

```
union(A, B): merged, new root = A
union(C, D): merged, new root = C
union(A, C): merged, new root = A
find(D) = A
union(B, H): already the same set (A)
final sets: A->A B->A C->A D->A E->A F->A G->A H->A
```

Union-find: rank + yol sıkıştırma



Bitti. Kümeler: {A,B,C,D,E,F,G,H}.

Karmaşıklık ve tuzak

- Her iki hile ile birlikte işlem başına $\sim O(1)$ amortize
- Tuzak: `union(a, b)` 'nin `a`, `b` 'yi doğrudan karşılaştırması
- **Kökler** birleştirilmeli — `find(a)`, `find(b)` — asla ham argümanlar değil

Kısa soru

Rank, bir kümedeki eleman sayısını değil, **yüksekliği** sayar. Neden doğrudan küme boyutu takip edilmiyor?

(cevap bir sonraki slaytta)

Cevap

Boyut da işe yarar ve yaygın bir alternatiftir — "boyuta göre birleştirme" küçük **kümeyi** büyüğün altına asar.

İkisi de aynı $O(\alpha(n))$ garantisini verir; rank, ağaç yüksekliğini doğrudan sınırladığı için klasik sunumdur — **find** 'ın gerçekten bedelini ödediği şey de budur.

4. En Küçük Yayılan Ağaçlar

Başlangıç sorusu

n kasabayı elektrik hatlarıyla bağlayın. Herhangi bir çift doğrudan bir maliyetle bağlanabilir.

Her şeyi hâlâ bağlayan en ucuz hat kümesi nedir?

Kısa tarihçe

- **Joseph Kruskal**, 1956
- **Robert Prim**, 1957 (Jarník'i, 1930, yeniden keşfederek)
- Aynı problem, yapısal olarak farklı iki açgözlü çözüm
- İkisi de kanıtlanabilir şekilde güvenli: **kesme özelliği** (cut property)

Kruskal'ın fikri

- Tüm kenarları ağırlığa göre sırala, bir kez
- En ucuzdan başlayarak tara; bir **döngü kapatmadıkça** kenarı ekle
- Döngü kontrolü: union-find, neredeyse $O(1)$
- Bağlı olmayan çizge $\rightarrow$ bir yayılan **orman**

Prim'in fikri

- Bir başlangıç köşesinden **tek** bir ağaç büyüt
- **İçeriden dışarıya** en ucuz kenarı ekle
- `key[v]` = v'yi ağaca şimdiye kadar bağlayan en ucuz kenar
- Farklı bir bileşene hiç ulaşmaz: key "sonsuz" kalır

Kruskal'ın kodu (C) — bölüm 1/2

```
int kruskal_mst(Edge *sorted, int edge_count,
               Edge *mst_out, int *total_out) {
    for (int v=0; v<vertex_count; v++)
        { parent_of[v]=v; rank_of[v]=0; }
    qsort(sorted, edge_count, sizeof(Edge), cmp_weight);
    int mst_len=0, total=0;
```

Kruskal'ın kodu (C) — bölüm 2/2

```
for (int i=0;i<edge_count;i++) {  
    if (find(sorted[i].a)==find(sorted[i].b))  
        continue;  
    union_sets(sorted[i].a, sorted[i].b);  
    mst_out[mst_len++]=sorted[i];  
    total+=sorted[i].w;  
}  
*total_out=total;  
return mst_len;  
}
```

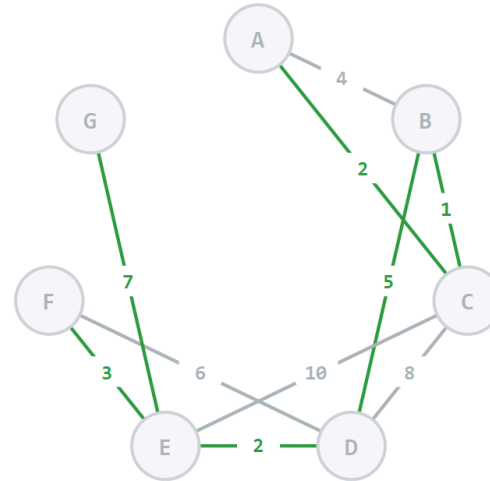
Kruskal'ın kodu (Java) — bölüm 1/2

```
static int kruskalMst(Edge[] sorted,
                    Edge[] mstOut, int[] totalOut) {
    for (int v=0;v<vertexCount;v++)
        { parentOf[v]=v; rankOf[v]=0; }
    Arrays.sort(sorted, Comparator.comparingInt(
        (Edge e) -> e.w).thenComparingInt(e -> e.idx));
    int mstLen=0, total=0;
```

Kruskal'ın kodu (Java) — bölüm 2/2

```
for (Edge e : sorted) {  
    if (find(e.a)==find(e.b)) continue;  
    unionSets(e.a, e.b);  
    mstOut[mstLen]=e; mstLen++;  
    total+=e.w;  
}  
totalOut[0]=total;  
return mstLen;  
}
```

Kruskal'ın MST'si



sıralı kenarlar =

B-C:1	A-C:2	D-E:2	E-F:3	A-B:4
B-D:5	D-F:6	E-G:7	C-D:8	C-E:10

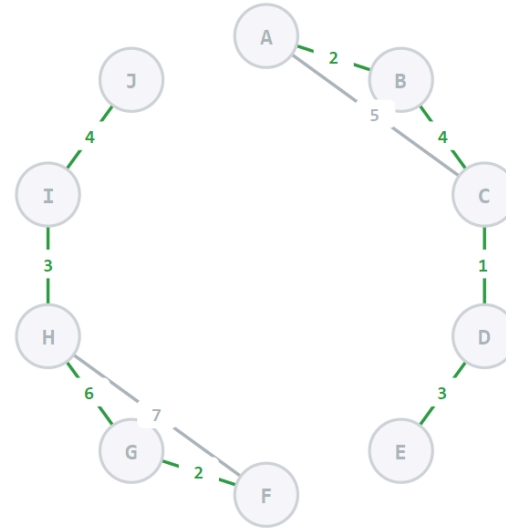
ebeveyn[] =

A	B	C	D	E	F	G
B	B	B	B	B	B	B

MST ağırlığı = 20

Bitti: 6 kenar seçildi, toplam ağırlık 20. Tek bir yayılan ağaç.

Uç durum: yayılan bir orman



sıralı kenarlar =

C-D:1	A-B:2	F-G:2	D-E:3	H-I:3	B-C:4
I-J:4	A-C:5	G-H:6	F-H:7		

ebeveyn[] =

A	B	C	D	E	F	G	H	I	J
A	A	A	C	C	F	F	F	H	H

MST ağırlığı = 25

Bitti: 8 kenar seçildi, toplam ağırlık 25. Çizge bağlı değildi: sonuç 2 bileşenli bir yayılan ORMAN.

Gerçek çıktı: Kruskal'ın MST'si

```
-- normal: 7 vertices, 10 edges, one component --  
MST edges: B-C:1 A-C:2 D-E:2 E-F:3 B-D:5 E-G:7  
total weight = 20  
components = 1  
  
-- edge: 10 edges, 2 components -- a spanning FOREST --  
MST edges: C-D:1 A-B:2 F-G:2 D-E:3 H-I:3 B-C:4 I-J:4 G-H:6  
total weight = 25  
components = 2 (a spanning forest)
```

Prim'in kodu (C) — bölüm 1/2

```
int prim_mst(Graph *g, int start,
             Edge *mst_out, int *total_out) {
    for (int v=0; v<g->vertex_count; v++)
        { key_of[v]=INF; in_mst[v]=0; parent_of[v]=-1; }
    key_of[start]=0;
    int mst_len=0, total=0;
    for (int count=0; count<g->vertex_count; count++) {
        int u=min_key_vertex(g->vertex_count);
        if (u==-1 || key_of[u]==INF) break;
        in_mst[u]=1;
```

Prim'in kodu (C) — bölüm 2/2

```
if (parent_of[u] != -1) {
    mst_out[mst_len].a = parent_of[u];
    mst_out[mst_len].b = u;
    mst_out[mst_len].w = key_of[u];
    mst_len++; total += key_of[u];
}
for (AdjNode *n = g->adj[u]; n; n = n->next)
    if (!in_mst[n->to] && n->weight < key_of[n->to])
        { key_of[n->to] = n->weight;
          parent_of[n->to] = u; }
}
*total_out = total;
return mst_len;
}
```

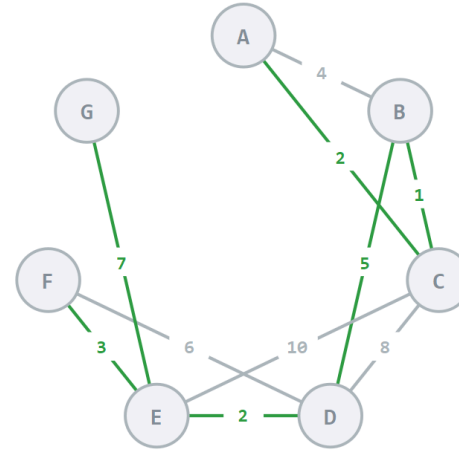
Prim'in kodu (Java) — bölüm 1/2

```
static int primMst(Graph g, int start,
                    Edge[] mstOut, int[] totalOut) {
    for (int v=0;v<g.vertexCount;v++)
        { keyOf[v]=INF; inMst[v]=false; parentOf[v]=-1; }
    keyOf[start]=0;
    int mstLen=0, total=0;
    for (int count=0;count<g.vertexCount;count++) {
        int u=minKeyVertex(g.vertexCount);
        if (u==-1 || keyOf[u]==INF) break;
        inMst[u]=true;
```

Prim'in kodu (Java) — bölüm 2/2

```
    if (parentOf[u] != -1) {
        mstOut[mstLen] = new Edge();
        mstOut[mstLen].a = parentOf[u];
        mstOut[mstLen].b = u;
        mstOut[mstLen].w = keyOf[u];
        mstLen++; total += keyOf[u];
    }
    for (AdjNode n = g.adj[u]; n != null; n = n.next)
        if (!inMst[n.to] && n.weight < keyOf[n.to])
            { keyOf[n.to] = n.weight; parentOf[n.to] = u; }
    }
    totalOut[0] = total;
    return mstLen;
}
```

Prim'in MST'si



anahtar[] =

A	B	C	D	E	F	G
0	1	2	5	2	3	7

ebeveyn[] =

-	C	A	B	D	E	E
---	---	---	---	---	---	---

öncelik kuyruğu =

--	--	--	--	--	--	--

MST ağırlığı = 20

Bitti: 6 kenar, toplam ağırlık 20.

Gerçek çıktı: Prim'in MST'si

```
-- normal: 7 vertices, 10 edges, starts at A --  
MST edges: A-C:2 C-B:1 B-D:5 D-E:2 E-F:3 E-G:7  
total weight = 20
```

```
-- edge: 2 components, starts at A -- F..J never reached --  
MST edges: A-B:2 B-C:4 C-D:1 D-E:3  
total weight = 10  
unreached: F G H I J
```

Karmaşıklık tablosu

Algoritma	Süre	Not
Kruskal	$O(E \log E)$	sıralama baskın
Prim (dizi)	$O(V^2)$	Dijkstra'nın şekliyle eşleşir
Prim (yığın)	$O(E \log V)$	yoğun çizgelerde daha iyi

Kısa soru

Prim, aynı çizgede Kruskal'dan **daha pahalı** bir kenar seçebilir mi?

(cevap: hayır — bir sonraki slayta bakın)

Cevap

Hayır. İkisi de kanıtlanabilir şekilde en iyisidir (kesme özelliği).

Bir çizgenin her MST'si aynı **toplam** ağırlığa sahiptir — ağırlıklar eşitlendiğinde sadece *hangi* kenarları seçtikleri farklılaşabilir.

5. Tek Kaynaktan En Kısa Yollar

Başlangıç sorusu

Mesafeye göre ağırlıklandırılmış bir yol ağı. Bir şehirden başlayarak,
her diğer şehre en ucuz ulaşım nasıl olur?

Kısa tarihçe

- Edsger Dijkstra, 1956 (1959'da yayımlandı)
- Yeni bir bilgisayarı tanıtmak için 20 dakikalık bir alıştırma
- Her ağırlık **negatif olmadığı**nda hâlâ standart cevap

Dijkstra'nın fikri

- `dist[v]` = şimdiye kadar bulunan en ucuz **toplam yol**
- Henüz bitmemiş, `dist` değeri en küçük köşeyi seç
- Seçildiğinde: **kesin**, bir daha asla küçülemez
- Bu sadece hiçbir kenar ağırlığı negatif olmadığı için doğru

Negatif ağırlıklar neden bozar

Erken "kesin" olarak çıkarılan bir köşe, daha sonra çok negatif bir kenardan geçen bir yolla yenilebilir — bu, köşe zaten kesinleştikten **sonra** keşfedilir.

Dijkstra'nın kodu (C) — bölüm 1/2

```
void dijkstra(Graph *g, int start) {  
    for (int v=0;v<g->vertex_count;v++)  
        { dist_of[v]=INF; done[v]=0; parent_of[v]=-1; }  
    dist_of[start]=0;  
    for (int count=0;count<g->vertex_count;count++) {  
        int u=min_dist_vertex(g->vertex_count);  
        if (u==-1 || dist_of[u]==INF) break;  
        done[u]=1;  
    }  
}
```

Dijkstra'nın kodu (C) — bölüm 2/2

```
    for (AdjNode *n=g->adj[u]; n; n=n->next) {  
        int cand=dist_of[u]+n->weight;  
        if (!done[n->to] && cand<dist_of[n->to])  
            { dist_of[n->to]=cand; parent_of[n->to]=u; }  
    }  
}
```

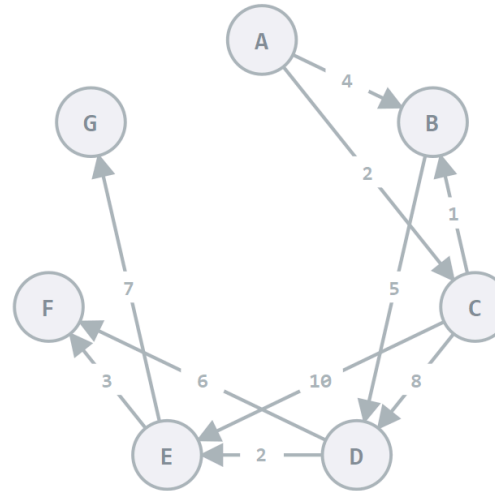
Dijkstra'nın kodu (Java) — bölüm 1/2

```
static void dijkstra(Graph g, int start) {  
    for (int v=0;v<g.vertexCount;v++)  
        { distOf[v]=INF; done[v]=false; parentOf[v]=-1; }  
    distOf[start]=0;  
    for (int count=0;count<g.vertexCount;count++) {  
        int u=minDistVertex(g.vertexCount);  
        if (u==-1 || distOf[u]==INF) break;  
        done[u]=true;  
    }  
}
```

Dijkstra'nın kodu (Java) — bölüm 2/2

```
    for (AdjNode n=g.adj[u]; n!=null; n=n.next) {  
        int cand=distOf[u]+n.weight;  
        if (!done[n.to] && cand<distOf[n.to])  
            { distOf[n.to]=cand; parentOf[n.to]=u; }  
    }  
}
```

Dijkstra en kısa yol



uzaklık[] =

A	B	C	D	E	F	G
0	3	2	8	10	13	17

ebeveyn[] =

-	C	A	B	D	E	E
---	---	---	---	---	---	---

öncelik kuyruğu =

--	--	--	--	--	--	--

Bitti. Uzaklıklar: A=0, B=3, C=2, D=8, E=10, F=13, G=17.

Gerçek çıktı: Dijkstra

```
-- normal: 8 vertices, 10 edges, starts at A --  
distances: A=0 B=3 C=2 D=8 E=10 F=13 G=17  
  
-- edge: F..J never reachable via the directed edges --  
distances: A=0 B=2 C=5 D=6 E=9 F=inf G=inf H=inf I=inf J=inf
```

Bellman-Ford: negatif ağırlıklar sorun değil

- Bellman ve Ford, 1950'lerin sonu
- "En küçüğü çıkar" fikrinden tamamen vazgeç
- Her kenarı, sabit sırayla, $V - 1$ tura kadar gevşet
- Hâlâ iyileştiren bir V . tur = bir negatif döngü

Bellman-Ford'un kodu (C) — bölüm 1/2

```
int bellman_ford(Graph *g, int start) {  
    for (int v=0;v<g->vertex_count;v++)  
        { dist_of[v]=INF; parent_of[v]=-1; }  
    dist_of[start]=0;  
    for (int p=1;p<=g->vertex_count-1;p++) {  
        int changed=0;  
        for (int u=0;u<g->vertex_count;u++) {  
            if (dist_of[u]==INF) continue;  

```

Bellman-Ford'un kodu (C) — bölüm 2/2

```
        for (AdjNode *n=g->adj[u]; n; n=n->next) {
            int cand=dist_of[u]+n->weight;
            if (cand<dist_of[n->to])
                { dist_of[n->to]=cand;
                  parent_of[n->to]=u; changed=1; }
        }
    }
    if (!changed) break;
}
/* one more pass detects a negative cycle */
}
```

Bellman-Ford'un kodu (Java) — bölüm 1/2

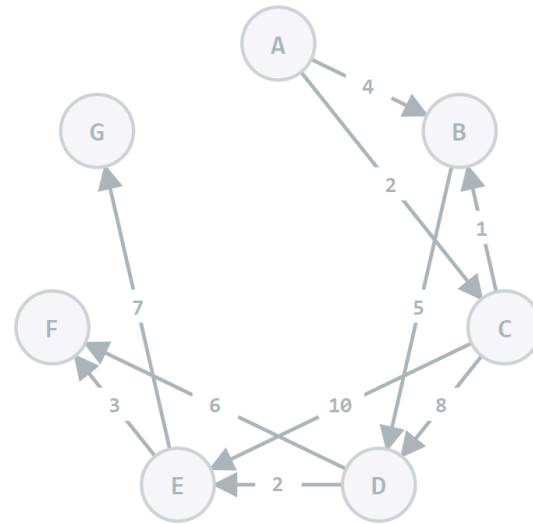
```
static boolean bellmanFord(Graph g, int start) {  
    for (int v=0;v<g.vertexCount;v++)  
        { distOf[v]=INF; parentOf[v]=-1; }  
    distOf[start]=0;  
    for (int pass=1;pass<=g.vertexCount-1;pass++) {  
        boolean changed=false;  
        for (int u=0;u<g.vertexCount;u++) {  
            if (distOf[u]==INF) continue;  

```

Bellman-Ford'un kodu (Java) — bölüm 2/2

```
        for (AdjNode n=g.adj[u]; n!=null; n=n.next) {
            int cand=distOf[u]+n.weight;
            if (cand<distOf[n.to])
                { distOf[n.to]=cand;
                  parentOf[n.to]=u; changed=true; }
        }
    }
    if (!changed) break;
}
/* one more pass detects a negative cycle */
}
```

Bellman-Ford en kısa yol



uzaklık[] =

A	B	C	D	E	F	G
0	3	2	8	10	13	17

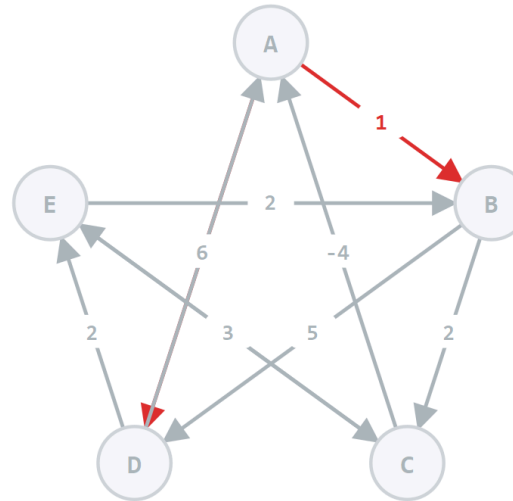
ebeveyn[] =

-	C	A	B	D	E	E
---	---	---	---	---	---	---

tur 3 / 6

Bitti. Uzaklıklar: A=0, B=3, C=2, D=8, E=10, F=13, G=17. Negatif döngü yok.

Zorunlu uç durum: negatif döngü



	A	B	C	D	E
uzaklık[] =	-4	-2	0	0	1

ebeveyn[] =	C	A	B	A	C
--------------	---	---	---	---	---

tur 4 / 4

Bitti. Uzaklıklar: A=-4, B=-2, C=0, D=0, E=1. Negatif döngü var.

Gerçek çıktı: Bellman-Ford

```
-- normal: 7 vertices, all positive, starts at A --  
distances: A=0 B=3 C=2 D=8 E=10 F=13 G=17  
no negative cycle
```

```
-- edge: A-B-C-A is a negative cycle (total -1) --  
distances: A=-4 B=-2 C=0 D=0 E=1  
negative cycle detected
```

Karmaşıklık tablosu

Algoritma	Süre	Negatifleri kaldırır mı?
Dijkstra	$O(V^2)$	Hayır
Bellman-Ford	$O(V \cdot E)$	Evet

Kısa soru

Bellman-Ford için neden tam olarak $V - 1$ tur, V veya $V/2$ değil?

(cevap bir sonraki slaytta)

Cevap

Bir en kısa yol (negatif döngü yoksa) bir köşeyi asla tekrarlamaz — en fazla $V - 1$ kenar.

Her tur, her yolun "kesinleşmiş" önekini bir kenar uzatır. $V - 1$ tur, mümkün olan en uzun en kısa yolu kesinleştirir.

6. Tüm Çiftler En Kısa Yollar

Başlangıç sorusu

Bir uçuş rezervasyon sistemi, ~20 şehrin **her çifti** arasında en ucuz ücrete aynı anda ihtiyaç duyar.

Dijkstra'yı 20 kez çalıştırmak işe yarar — işi **paylaşmanın** bir yolu var mı?

Kısa tarihçe

- **Robert Floyd** ve **Stephen Warshall**, ikisi de 1962
- Bağımsız, yakından ilişkili matris algoritmaları
- Birleşik algoritma her iki adı da taşır

Fikir

- Bir $N \times N$ matris `dist[i][j]`
- **Her** köşe `k` için: `i -> k -> j`, `dist[i][j]` 'den daha mı kısa?
- Her köşeyi bir ara durak olarak dene
- **Yerinde** güncelle — güvenli, çünkü `dist[i][k] / dist[k][j]` geçiş sırasında hiç değişmez

Floyd-Warshall'ın kodu (C)

```
void floyd_warshall(int vertex_cnt) {  
    for (int k=0;k<vertex_cnt;k++) {  
        for (int i=0;i<vertex_cnt;i++) {  
            for (int j=0;j<vertex_cnt;j++) {  
                if (dist[i][k]==INF || dist[k][j]==INF)  
                    continue;  
                int through=dist[i][k]+dist[k][j];  
                if (through<dist[i][j])  
                    dist[i][j]=through;  
            }  
        }  
    }  
}
```

Floyd-Warshall'ın kodu (Java)

```
static void floydWarshall(int vertexCnt) {  
    for (int k=0;k<vertexCnt;k++) {  
        for (int i=0;i<vertexCnt;i++) {  
            for (int j=0;j<vertexCnt;j++) {  
                if (dist[i][k]==INF || dist[k][j]==INF)  
                    continue;  
                int through=dist[i][k]+dist[k][j];  
                if (through<dist[i][j])  
                    dist[i][j]=through;  
            }  
        }  
    }  
}
```

Floyd-Warshall: matris dolar

	A	B	C	D	E
A	0	1	-3	2	-4
B	3	0	-4	1	-2
C	7	4	0	5	2
D	2	-1	-5	0	-3
E	8	5	1	6	0

Bitti: 5 düğümün her çifti için en kısa yol hesaplandı. Negatif döngü yok.

Gerçek çıktı: Floyd-Warshall

```
-- normal: 5 vertices, negative edges but no negative cycle --
```

	A	B	C	E	D
A	0	1	-3	-4	2
B	3	0	-4	-2	1
C	7	4	0	2	5

```
no negative cycle
```

```
-- edge: A-B-C-A is a negative cycle --
```

```
negative cycle at: A B C
```

Karmaşıklık ve tuzak

- $O(V^3)$ süre, $O(V^2)$ alan
- Birkaç yüz köşe için pratik, daha fazlası için değil
- Tuzak: `k`'yı **en içteki** döngü yapmak sessizce çöp hesaplar
- Tuzak: `== INF` korumasını unutmak → tamsayı taşması

Kısa soru

Bellman-Ford, **bir başlangıçtan** erişilebilen negatif döngüleri bulur. Floyd-Warshall'ın köşegen kontrolü bunları farklı nasıl bulur?

(cevap bir sonraki slaytta)

Cevap

Sıfırın altına düşen herhangi bir `dist[v][v]` : `v` 'den ayrılan bir yolun, yerinde kalmaktan daha ucuza geri döndüğü anlamına gelir.

Floyd-Warshall **her** çifti hesapladığı için, bu kontrol her köşe için aynı anda çalışır — ayrı bir başlangıç köşesine gerek yok.

7. Güçlü Bağlı Bileşenler

Başlangıç sorusu

Kenar yönüne saygı göstererek hangi köşe grupları birbirine ulaşıp **geri dönebilir**?

Bir döngü içinde birbirine bağlanan web sayfaları. Karşılıklı özyinelemeli fonksiyonlar.

Kosaraju'nun fikri

- **Sergei Kosaraju**, ~1978 (Micali ve Vazirani, 1981 üzerinden atfedilir)
- Aşama 1: DFS, her köşenin **bitiş zamanını** kaydet
- Aşama 2: **Devrik** (kenarları ters çevrilmiş) çizgede DFS
- Kökleri **azalan** bitiş zamanı sırasıyla ziyaret et
- Aşama 2'deki her DFS ağacı = bir GBB (güçlü bağlı bileşen)

Kosaraju'nun kodu (C)

```
void dfs1(int u) {           /* phase 1 */
    visited[u]=1;
    for (AdjNode *n=cur_g->adj[u]; n; n=n->next)
        if (!visited[n->to]) dfs1(n->to);
    finish[finish_len++]=u;
}

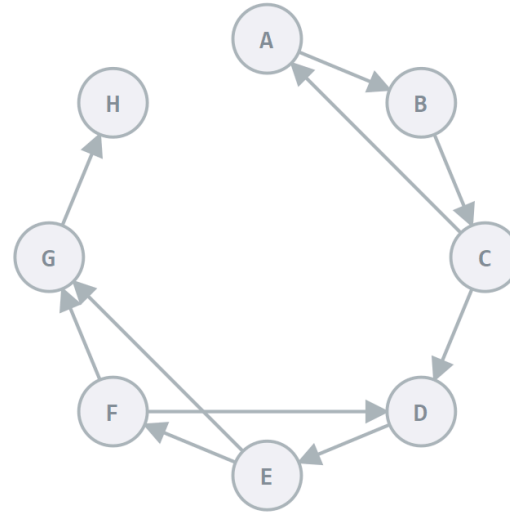
void dfs2(int u, int id) {    /* phase 2, on adjT */
    visited[u]=1; comp_of[u]=id;
    for (AdjNode *n=cur_g->adjT[u]; n; n=n->next)
        if (!visited[n->to]) dfs2(n->to, id);
}
```

Kosaraju'nun kodu (Java)

```
static void dfs1(int u) {           // phase 1
    visited[u]=1;
    for (AdjNode n=curG.adj[u]; n!=null; n=n.next)
        if (visited[n.to]==0) dfs1(n.to);
    finish[finishLen]=u; finishLen++;
}

static void dfs2(int u, int id) {    // phase 2, on adjT
    visited[u]=1; compOf[u]=id;
    for (AdjNode n=curG.adjT[u]; n!=null; n=n.next)
        if (visited[n.to]==0) dfs2(n.to, id);
}
```

Güçlü bağlı bileşenler



çağrı yığını =

--	--	--	--	--	--	--	--

bitiş sırası =

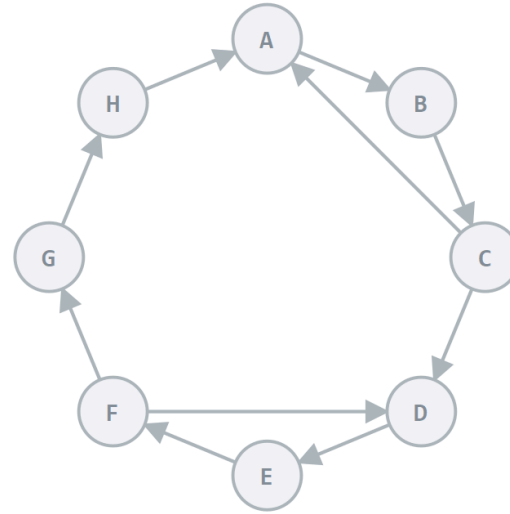
H	G	F	E	D	C	B	A
---	---	---	---	---	---	---	---

bileşen[] =

A	B	C	D	E	F	G	H
0	0	0	1	1	1	2	3

Biti: 4 güçlü bağlı bileşen bulundu: {A,B,C} {D,E,F} {G} {H}

Uç durum: tek bir büyük döngü



çağrı yığını =

--	--	--	--	--	--	--	--

bitiş sırası =

H	G	F	E	D	C	B	A
---	---	---	---	---	---	---	---

bileşen[] =

A	B	C	D	E	F	G	H
0	0	0	0	0	0	0	0

Biti: 1 güçlü bağlı bileşen bulundu: {A,B,C,D,E,F,G,H}

Gerçek çıktı: güçlü bağlı bileşenler

```
-- normal: 8 vertices, 2 cyclic components + 2 singletons --  
4 components:  
  A B C  
  D E F  
  G  
  H  
  
-- edge: everything is one big cycle --  
1 component:  
  A B C D E F G H
```

Karmaşıklık ve tuzak

- $O(V + E)$ — iki tam DFS geçişi
- Tuzak: aşama 2'yi aşama 1 ile **aynı** sırada çalıştırmak
- **Tersine çevrilmeli** — buradaki en yaygın hata

Kısa soru

Bir **DAG**'da (hiç döngü yok), Kosaraju'nun algoritması kaç güçlü bağlı bileşen bulur?

(cevap bir sonraki slaytta)

Cevap

Tam olarak **V** — köşe başına bir tane.

Hiçbir yerde döngü olmadığında, hiçbir köşe herhangi bir yoldan kendine dönemez, bu yüzden her bileşen tek başınadır (singleton). Bu, bu haftaki GBB animasyonundaki son uç durumdur.

8. İki Parçalı Çizgeler

Başlangıç sorusu

Sınavları hiçbir öğrencinin aynı anda iki sınavı olmayacak şekilde planlayın. Bir öğrenciyi paylaşan dersler → bir kenar.

Bu çizge 2 renklenebilir mi — mümkün olan en basit ders programı?

İki renkli BFS

- Başlangıcı $\emptyset$ renklendir; her komşuyu **karşıt** renk yap
- Zaten kuyrukta olan aynı renkli bir komşu mu? **Çelişki** — iki parçalı değil
- Bileşen başına bir BFS (Hafta 5'in bileşenler döngüsü gibi)

Tek sayılı döngü denkliği

Bir çizge, ancak ve ancak tek uzunluklu bir döngüsü yoksa iki parçalıdır.

Çift bir döngü renkleri kusursuzca değiştirir. Tek bir döngü tutarlı bir şekilde kapanamaz.

İki parçalılık kontrolü kodu (C) — bölüm 1/2

```
int is_bipartite(Graph *g) {  
    for (int i=0; i<g->vertex_count; i++) color_of[i]=-1;  
    for (int s=0; s<g->vertex_count; s++) {  
        if (color_of[s]!=-1) continue;  
        color_of[s]=0; front=rear=0; enqueue(s);  
        while (front<rear) {  
            int u=dequeue();  
            // ... (rest of the code for this block)  
        }  
    }  
}
```

İki parçalılık kontrolü kodu (C) — bölüm 2/2

```
    for (AdjNode *n=g->adj[u]; n; n=n->next) {
        if (color_of[n->to]==-1)
            { color_of[n->to]=1-color_of[u];
              enqueue(n->to); }
        else if (color_of[n->to]==color_of[u])
            return 0;
    }
}
return 1;
}
```

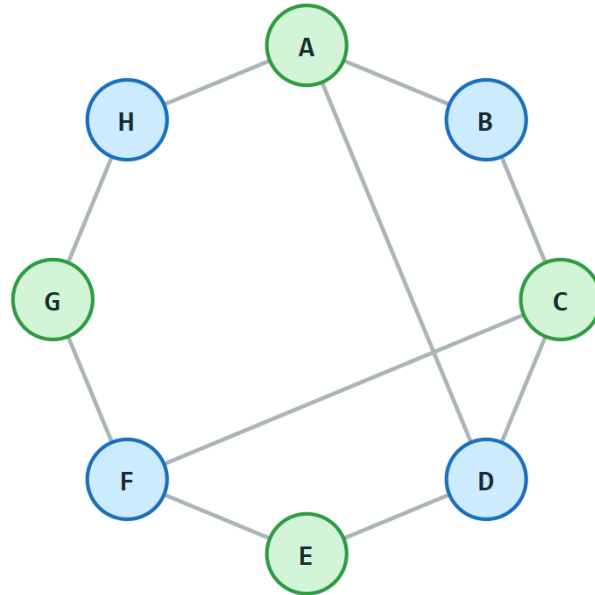
İki parçalılık kontrolü kodu (Java) — bölüm 1/2

```
static boolean isBipartite(Graph g) {  
    for (int i=0;i<g.vertexCount;i++) colorOf[i]=-1;  
    for (int s=0;s<g.vertexCount;s++) {  
        if (colorOf[s]!=-1) continue;  
        colorOf[s]=0; front=rear=0; enqueue(s);  
        while (front<rear) {  
            int u=dequeue();
```

İki parçalılık kontrolü kodu (Java) — bölüm 2/2

```
        for (AdjNode n=g.adj[u]; n!=null; n=n.next) {  
            if (colorOf[n.to]==-1)  
                { colorOf[n.to]=1-colorOf[u];  
                  enqueue(n.to); }  
            else if (colorOf[n.to]==colorOf[u])  
                return false;  
        }  
    }  
    return true;  
}
```

İki parçalı çizge kontrolü



renk[] =

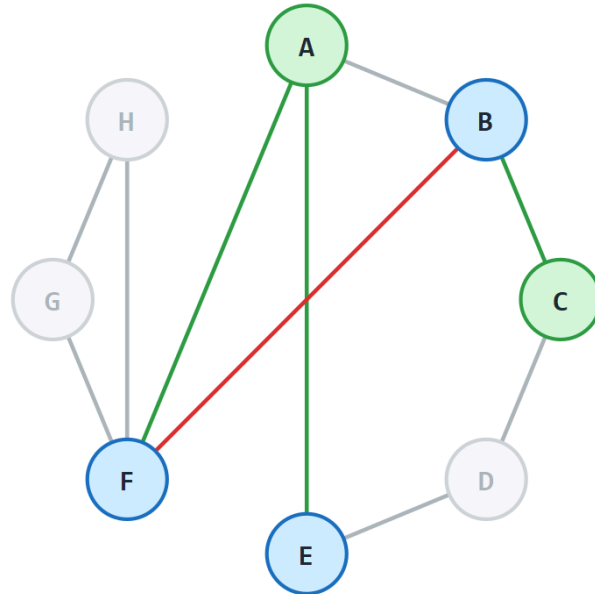
A	B	C	D	E	F	G	H
0	1	0	1	0	1	0	1

kuyruk =

--	--	--	--	--	--	--	--

Bitti. İki parçalı: renk 0 ve renk 1 tarafları birbirine hiç komşu değil.

Uç durum: tek sayılı bir döngü



renk[] =

A	B	C	D	E	F	G	H
0	1	0	-	1	1	-	-

kuyruk =

E	F	C					
---	---	---	--	--	--	--	--

Bitti. İki parçalı değil: tek sayılı bir döngü var.

Gerçek çıktı: iki parçalılık kontrolü

```
-- normal: an even cycle plus 2 safe diagonals --  
colors: A=0 B=1 C=0 D=1 E=0 F=1 G=0 H=1  
bipartite  
  
-- edge: A-B-C-D-E-A is a 5-cycle (odd) --  
colors: A=0 B=1 C=0 D=-1 E=1 F=1 G=-1 H=-1  
NOT bipartite
```

Karmaşıklık ve tuzak

- $O(V + E)$ — bir BFS
- Tuzak: **bağlı olmayan** çizgeleri ele almamak
- İkinci bir bileşende izole bir tek sayılı döngü, bileşen başına döngü olmadan görünmez kalır

Kısa soru

Neden bir **çelişki kenarı** ("komşusuyla aynı renk") her zaman tek bir BFS katmanı içinde veya iki komşu katman arasında ortaya çıkar — asla bir katman atlamaz?

(cevap bir sonraki slaytta)

Cevap

BFS, kaynaktan **mesafe tek/çift durumuna** göre kesin olarak renklendirir — çift mesafe 0 rengini, tek mesafe 1 rengini alır.

Herhangi bir kenar, BFS mesafeleri tam olarak 0 veya 1 farklı olan köşeleri bağlar (asla 2+ değil, yoksa doğrudan bir kenar olmazdı) — bu yüzden bir çelişki tam olarak orada ortaya çıkabilir.

9. En Büyük Akış

Başlangıç sorusu

Bir su ağı: kaynak, hedef, kapasiteli borular.

Ağın aynı anda taşıyabileceği en büyük toplam akış nedir?

Kısa tarihçe

- Ford ve Fulkerson, 1956 — genel yöntem
- Edmonds ve Karp, 1972 — her zaman **en kısa** genişletme yolunu kullan
- Kanıtlandı: bu seçim polinom zaman garantiler

Fikir

- Boş kapasitesi olan, s'den t'ye herhangi bir **genişletme yolu** bul
- **Darboğazı** it (yol üzerindeki en küçük kapasite)
- İleri itmek bir **ters** kenar açar — sonraki bir yol bunu "geri alabilir"
- "Hâlâ kullanılabilir" çizgesi, **artık (residual) çizgedir**

En büyük akış - en küçük kesim

En büyük akış, en ucuz kesime **eşittir** — s 'yi t 'den ayıran en küçük toplam kapasite.

Geriye genişletme yolu kalmaması = s 'den BFS'in erişilebilir kümesi, o kesimin **kendisidir**.

Edmonds-Karp kodu (C) — bölüm 1/2

```
int edmonds_karp(int vertex_cnt, int s, int t) {  
    int max_flow=0;  
    while (bfs_augmenting_path(vertex_cnt, s, t)) {  
        int bottleneck=INF;  
        for (int v=t; v!=s; v=parent_of[v])  
            if (cap_of[parent_of[v]][v]<bottleneck)  
                bottleneck=cap_of[parent_of[v]][v];  
        max_flow+=bottleneck;  
    }  
    return max_flow;  
}
```

Edmonds-Karp kodu (C) — bölüm 2/2

```
    for (int v=t; v!=s; v=parent_of[v]) {  
        int u=parent_of[v];  
        cap_of[u][v]-=bottleneck;  
        cap_of[v][u]+=bottleneck;  
    }  
    max_flow+=bottleneck;  
}  
return max_flow;  
}
```

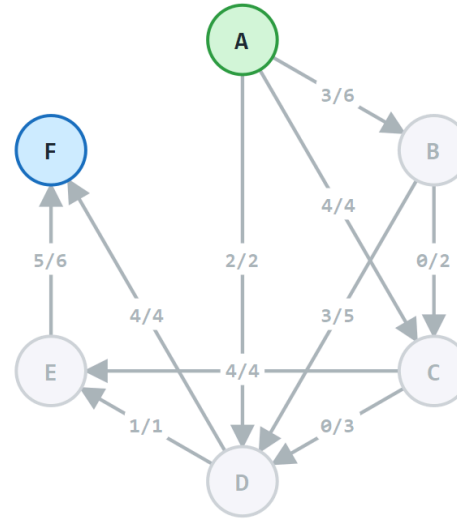
Edmonds-Karp kodu (Java) — bölüm 1/2

```
static int edmondsKarp(int vertexCnt, int s, int t) {  
    int maxFlow=0;  
    while (bfsAugmentingPath(vertexCnt, s, t)) {  
        int bottleneck=INF;  
        for (int v=t; v!=s; v=parentOf[v]) {  
            int u=parentOf[v];  
            if (capOf[u][v]<bottleneck)  
                bottleneck=capOf[u][v];  
        }  
    }
```

Edmonds-Karp kodu (Java) — bölüm 2/2

```
    for (int v=t; v!=s; v=parentOf[v]) {  
        int u=parentOf[v];  
        capOf[u][v]-=bottleneck;  
        capOf[v][u]+=bottleneck;  
    }  
    maxFlow+=bottleneck;  
}  
return maxFlow;  
}
```

Edmonds-Karp en büyük akış



kalan (residual) =

A>B:3	B>A:3	B>C:2	B>D:2	C>A:4
C>D:3	D>A:2	D>B:3	E>C:4	E>D:1
E>F:1	F>D:4	F>E:5		

en büyük akış = 9

Bitti. A'dan F'ye en büyük akış = 9 (4 artırıcı yol kullanıldı).

Gerçek çıktı: Edmonds-Karp

```
-- normal: 6 vertices, 10 edges, A to F --  
max flow from A to F = 9  
  
-- edge: A and J are in two separate components --  
max flow from A to J = 0
```

Karmaşıklık ve tuzak

- Özellikle Edmonds-Karp için $O(V * E^2)$
- Tuzak: **ters** kenar güncellemesini unutmak
- Onsuz: sade Ford-Fulkerson, en iyi sonuçtan kısa kalabilir

Kısa soru

Bölüm 3'ün union-find'ı "0 akış, s ve t bağlı değil" cevabını **anında** verebilirdi. Bu bölüm neden hâlâ önce tam bir BFS çalıştırıyor?

(cevap bir sonraki slaytta)

Cevap

Union-find sadece **erişilebilirliği** bilir, **kapasiteyi** değil.

s ve t bağlı OLSA bile, gerçek en büyük akış yol boyunca darboğaz kapasitelerine bağlıdır — union-find'ın ebeveyn işaretçilerinin hiç kaydetmediği bir şey.

10. Geri İzleme

Başlangıç sorusu

Bir haritayı, hiçbir iki komşu aynı rengi paylaşmayacak şekilde, mümkün olduğunca az renkle boyayın.

Bilinen bir formül yok. Sistematik olarak nasıl arama yaparsınız?

Fikir: dene, özyinele, geri al

- Kısmi bir çözümü bir kararla genişlet
- Hâlâ tutarlı mı? **Özyinele** ve genişletmeye devam et
- Tutarsız mı, veya her genişletme başarısız mı? **Geri al** ve bir sonraki seçeneği dene

Neden çizge boyama, Hamilton yolu değil?

Boyama, Bölüm 8'in tam `color[]` satırını ve komşu-çelişki kontrolünü yeniden kullanır.

Bir Hamilton yolu, görece az ekstra kavrayış için tamamen yeni bir "şimdiye kadarki yol" kuralına ihtiyaç duyar.

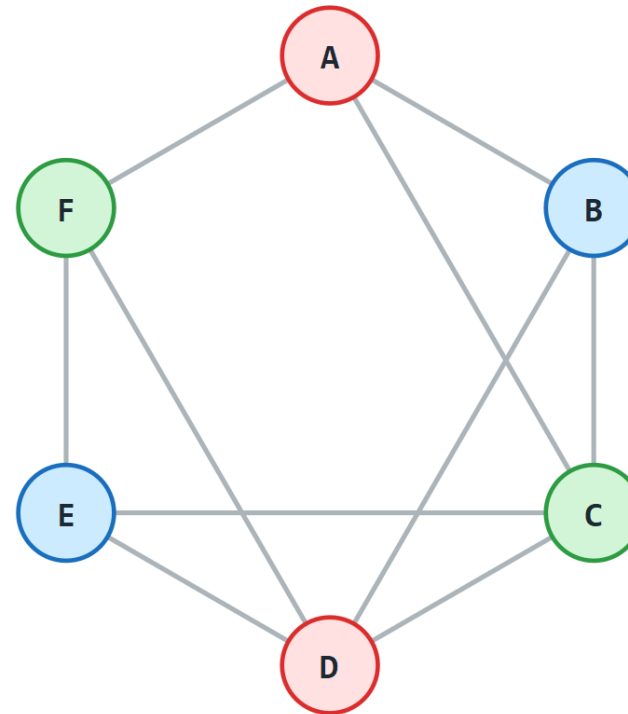
Geri izleme kodu (C)

```
int color_graph(int v, int k) {  
    if (v==cur_g->vertex_count) return 1;  
    for (int c=1;c<=k;c++) {  
        if (safe(v,c)) {  
            color_of[v]=c;  
            if (color_graph(v+1,k)) return 1;  
            color_of[v]=0; /* backtrack */  
        }  
    }  
    return 0;  
}
```

Geri izleme kodu (Java)

```
static boolean colorGraph(int v, int k) {  
    if (v==curG.vertexCount) return true;  
    for (int c=1;c<=k;c++) {  
        if (safe(v,c)) {  
            colorOf[v]=c;  
            if (colorGraph(v+1,k)) return true;  
            colorOf[v]=0;    // backtrack  
        }  
    }  
    return false;  
}
```

Geri izleme: çizge boyama

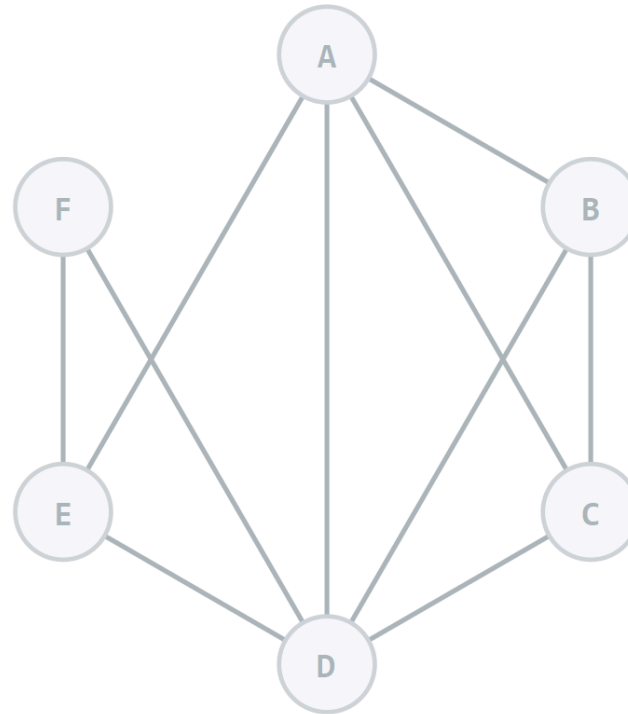


renk[] =

A	B	C	D	E	F
1	2	3	1	2	3

Bitti. Boyama: A=1, B=2, C=3, D=1, E=2, F=3.

Zorunlu uç durum: K4, çözülemez



renk[] =

A	B	C	D	E	F
0	0	0	0	0	0

Bitti. 3 renk yetmiyor.

Gerçek çıktı: geri izlemeli çizge boyama

```
-- normal: 6 vertices, 10 edges, k=3 --  
colouring: A=1 B=2 C=3 D=1 E=2 F=3  
  
-- edge: K4 is UNSOLVABLE with k=3 --  
no valid colouring with k=3
```

Karmaşıklık ve tuzak

- En kötü durum $O(k^V)$ — son çare bir teknik
- `safe()` erkenden büyük kısımları budar — kullanılabilir olmasının sebebi bu
- Tuzak: **geri alma** adımını unutmak sonraki dalları bozar

Kısa soru

K4, 4 renk gerektirir. Algoritma, $k=3$ ile "çözülemez" bildirmeden önce gerçekten kaç **köşeyi** boyamaya çalışır?

(cevap bir sonraki slaytta)

Cevap

Her seferinde köşe 0'ın ötesine geri izleme yaptığında hepsini — ama başarısızlık köşe 3'te, K4'ün 4. köşesinde tespit edilir, çünkü ilk 3'ü aralarında zaten 3 mevcut rengin tamamını kullanmıştır.

`safe()` ardından köşe 3 için her rengi reddeder, başa kadar bir geri alma zinciri zorlar.

Her program nasıl kontrol edildi

- Her C programı `-Wall -Wextra -Werror` ile derlenir
- Her Java programı `-Xlint:all -Werror` ile derlenir
- C ve Java çıktısı **bayt bayt özdeş** olarak karşılaştırıldı, aynı senaryolar
- Her birim test dosyası **AddressSanitizer** altında da yeniden derlendi

Algoritma başına tek cümle

- **Kahn / DFS topolojik sıralama:** her "önce" kuralına uyan sıra
- **Döngü tespiti:** sadece var mı değil, hangi köşeler
- **Union-find:** "aynı grup mu?" sorusu neredeyse sabit zamanda
- **Kruskal / Prim:** her şeyi bağlamanın en ucuz yolu
- **Dijkstra / Bellman-Ford:** bir başlangıçtan en ucuz yol
- **Floyd-Warshall:** her çift arasındaki en ucuz yol, aynı anda

Algoritma başına tek cümle (devam)

- **Kosaraju GBB:** karşılıklı erişilebilirlik, yönlü
- **İki parçalılık kontrolü:** bu tam olarak iki takıma bölünebilir mi?
- **Edmonds-Karp:** bir ağın taşıyabileceği en büyük akış
- **Geri izleme:** dene, özyinele, geri al — formülü olmayan problemler için

Özet tablosu (1/3)

Problem	Algoritma	Süre
Sıralama	Kahn / DFS topo-sıralama	$O(V+E)$
Döngü var mı?	3 renkli DFS	$O(V+E)$
Gruplar	Union-find	$\sim O(1)$ amortize

Özet tablosu (2/3)

Problem	Algoritma	Süre
En ucuz bağlantı	Kruskal / Prim	$O(E \log E) / O(V^2)$
En ucuz yol	Dijkstra / Bellman-Ford	$O(V^2) / O(V \cdot E)$
Tüm çiftler yollar	Floyd-Warshall	$O(V^3)$

Özet tablosu (3/3)

Problem	Algoritma	Süre
Karşılıklı erişim	Kosaraju GBB	$O(V+E)$
2 takıma bölme	İki parçalılık kontrolü	$O(V+E)$
En büyük akış	Edmonds-Karp	$O(V \cdot E^2)$
Formül yok	Geri izleme	En kötü $O(k^V)$

Alıştırmalar (birkaçını seçin)

- Kahn'ın algoritmasını 5 kenarlı bir DAG üzerinde elle izleyin
- Negatif bir kenarı olan ama negatif döngüsü olmayan bir çizge kurun
- GBB'yi önemsiz (tek köşeli) bileşenleri işaretleyecek şekilde değiştirin
- Her ağacın iki parçalı olduğunu kanıtlayın

Kendini sinama: 3 hızlı soru

1. Dijkstra neden negatif bir kenarda başarısız olur?
2. Kosaraju'nun topolojik-sıralama-DFS'e kattığı TEK yeni fikir nedir?
3. Ters bir artık kenar, sonraki bir genişletme yoluna ne yapmasına izin verir?

Sırada ne var

Hafta 10: İleri Ağaç Yapıları

AVL ağaçları, kırmızı-siyah ağaçlar, B-ağaçları — talihsiz eklemelerde $O(\log n)$ 'in $O(n)$ 'e çökmesini önleyen kendi kendini dengeleyen makine.

Sorular?

CEN207 Veri Yapıları · Hafta 9 · Çizge Algoritmaları